SYSTEM RENOVIERUNG

OpenClaw AI Harness

Von 175k LOC "Frankenstein" zu modularem, atomarem Multi-Agent-System mit Rust/PowerShell-Hybrid-Architektur

Version: 1.0

Datum: 7. April 2026

Autor: andre (GitHub: andre-bubel)

Status: Phase 0 abgeschlossen

Inhaltsverzeichnis

1. Executive Summary
2. Ausgangssituation & Probleme
3. Ziel-Architektur
4. Phase 0: Stabilisierung (ABGESCHLOSSEN)
5. Phase 1: Modularisierung
6. Phase 2: State Management
7. Phase 3: Obsidian RAG
8. Phase 4: Code-Analyse
9. Phase 5: Sequential Workflow
10. Phase 6: Integration & Polishing
11. Zusammenfassung & Naechste Schritte

1. Executive Summary

175k
Lines of Code

23
Dateien repariert

31
Operationen ersetzt

Dieser Plan dokumentiert die systematische Renovierung des OpenClaw AI Harness Systems von einer instabilen "Frankenstein"-Codebasis (175k LOC aus kombinierten Projekten) zu einem modularen, robusten Multi-Agent-System mit atomaren Dateioperationen.

> **Was wurde erreicht (Phase 0)**
> - Atomare Schreiboperationen implementiert (Rust-Core)
> - 23 PowerShell-Skripte von gefaehrlichen Set-Content/Out-File auf Safe-FileWrite umgestellt
> - Automatische Backup-Erstellung bei jedem Ueberschreiben
> - Sicheres Loeschen mit .deleted-Ordner statt permanentem Delete

Projekt-Uebersicht

Tabelle 1 Projekt-Metriken

Metrik	Wert
Repository	https://github.com/andre-bubel/openclaw
Lokaler Pfad	C:\Users\andre\.openclaw
Workspace	C:\Users\andre\.openclaw\workspace
Binary	target\release\openclaw-core.exe
Technologie-Stack	Rust (Core) + Python (AI) + PowerShell (Automation)

2. Ausgangssituation & Probleme

2.1 Codebasis-Herkunft

Das System entstand durch Kombination zweier grosser Open-Source-Projekte:

- **everything-claude-code** (142k GitHub Stars)
- **claw-code** (173k GitHub Stars)

Resultat: 175k Lines of Code ohne klare Architektur.

2.2 Kritische Probleme

Tabelle 2 Identifizierte Probleme

Problem	Auswirkung	Prioritaet
Dateikorruption bei Ueberschreiben	Session-Dateien (JSONL) gingen verloren	KRITISCH
Keine atomaren Schreiboperationen	Crash waehrend Write = kaputte Datei	KRITISCH
Keine Backups	Keine Wiederherstellung moeglich	Hoch
Mischung ohne Architektur	Rust/Python/PowerShell unstrukturiert	Mittel
Keine Modularisierung	Aenderungen haben Seiteneffekte	Mittel

Hauptrisiko: Bei einem Crash waehrend des Schreibens einer Session-Datei war die Datei unbrauchbar. Dies passierte wiederholt und fuehrte zu Datenverlust.

3. Ziel-Architektur

3.1 Hybrid-Architektur

Rust Core - Atomare Dateioperationen, Performance

Python AI Layer - Embeddings, Agents, LLM-Integration

PowerShell Automation - Windows-Integration, Session-Handling

Markdown + YAML - PARA-Struktur fuer Obsidian

3.2 PARA-Struktur (Second Brain)

```
.openclaw/
├── 00-Dashboard/
│   └── Dashboard.md              # Haupt-Uebersicht
├── 01-Sessions/
│   └── 2026-04-07-session.md     # Markdown + YAML Frontmatter
├── 02-Areas/
│   ├── Code-Analysis/
│   ├── Documentation/
│   └── Refactoring/
├── 03-Resources/
│   ├── RAG-Index/
│   └── Codebase-Analysis/
└── 04-Archive/
    └── (alte Sessions)
```

3.3 Phasen-Uebersicht

Tabelle 3 Renovierungsphasen

Phase	Name	Status	Ziel
0	Stabilisierung	ABGESCHLOSSEN	Atomic Writes, keine Korruption
1	Modularisierung	OFFEN	Cargo Workspace mit mehreren Crates
2	State Management	OFFEN	Markdown + YAML, Auto-Commit
3	Obsidian RAG	OFFEN	LanceDB + BGE-M3 Embeddings
4	Code-Analyse	OFFEN	Tree-sitter + Multi-Agent-System
5	Sequential Workflow	OFFEN	Orchestrator (nicht parallel)

6	Integration	OFFEN	CLI + Web-Service

4. Phase 0: Stabilisierung (ABGESCHLOSSEN)

0 ABGESCHLOSSEN - 7. APRIL 2026

4.1 Ziel

Verhindern von Dateikorruption durch atomare Schreiboperationen. Jede Dateioperation muss entweder vollstaendig erfolgreich sein oder gar nicht stattfinden - nie einen Zwischenzustand hinterlassen.

4.2 Implementierte Komponenten

4.2.1 Rust Core (atomic.rs)

```
pub fn atomic_write<P: AsRef<Path>>(path: P, content: &str) -> Result<()>
pub fn safe_delete<P: AsRef<Path>>(path: P) -> Result<()>
```

Algorithmus:

1. Backup erstellen (falls Datei existiert) mit Zeitstempel
2. In Temp-Datei schreiben (Datei.tmp)
3. fs::sync_all() - Sicheres Flush auf Disk
4. Atomares fs::rename() (kann nicht unterbrochen werden)

4.2.2 CLI-Binary

```
openclaw-core write <DATEI> <INHALT>   # Atomares Schreiben
openclaw-core delete <DATEI>           # Sicheres Loeschen
```

4.2.3 PowerShell Wrapper

Tabelle 4 PowerShell-Module

Modul	Funktion	Pfad
Safe-FileWrite.psm1	Atomares Schreiben mit Pipeline-Support	workspace/scripts/
Safe-Delete.psm1	Sicheres Loeschen nach .deleted/	workspace/scripts/

4.3 Durchgefuehrte Arbeiten

Tabelle 5 Phase 0 Ergebnisse

Metrik	Wert
Reparierte Dateien	23
Ersetzte Operationen	31
Erstellte Backups	23 (je Datei ein Backup)
Git Commit	5ce5ea0
Geaenderte Dateien im Commit	77

4.4 Kritische Dateien repariert

- **Convert-Sessions.ps1** - Session-Konvertierung JSONL zu Markdown
- **Sync-AutoresearchToObsidian.ps1** - Obsidian-Synchronisation
- **sync-openclaw-to-secondbrain.ps1** - SecondBrain-Sync
- **cmd-checkpoint.ps1** - Checkpoint-Erstellung
- **drift-check.ps1** - Drift-Detection

5. Phase 1: Modularisierung

1 OFFEN

5.1 Ziel

Aufbau einer modularen Cargo-Workspace-Struktur mit separaten Crates fuer verschiedene Funktionsbereiche.

5.2 Geplante Crate-Struktur

```
workspace/
├── Cargo.toml                          # Workspace-Root
└── crates/
    ├── openclaw-core/                  # Bereits vorhanden
    │   ├── src/atomic.rs               # Atomare Operationen
    │   ├── src/state.rs                # State Management
    │   └── src/rag.rs                  # RAG-Grundlagen
    ├── openclaw-agents/                # NEU: Agent-System
    │   ├── src/orchestrator.rs         # Workflow-Orchestrierung
    │   ├── src/code_analyzer.rs        # Code-Analyse-Agent
    │   ├── src/doc_agent.rs            # Dokumentations-Agent
    │   ├── src/security_agent.rs       # Security-Agent
    │   └── src/refactor_agent.rs       # Refactoring-Agent
    ├── openclaw-rag/                   # NEU: RAG-System
    │   ├── src/embeddings.rs           # BGE-M3 Integration
    │   ├── src/vectordb.rs             # LanceDB Wrapper
    │   └── src/indexer.rs              # Vault-Indexer
    └── openclaw-parser/                # NEU: Code-Parsing
        ├── src/treesitter.rs           # Tree-sitter Integration
        └── src/ast_analysis.rs         # AST-Analyse
```

5.3 Schritte

1. Neue Crates mit `cargo new --lib` erstellen
2. Workspace-Cargo.toml erweitern
3. Abhaengigkeiten zwischen Crates definieren
4. Core-Funktionalitaet in openclaw-core belassen
5. Agent-Logik in openclaw-agents verschieben

6. Phase 2: State Management

2 OFFEN

6.1 Ziel

Umstellung von JSONL-Sessions auf Markdown mit YAML-Frontmatter. Automatisches Git-Commit nach jeder Aenderung.

6.2 Session-Format

Vorher (JSONL)

```
{"timestamp": "2026-04-07T10:00:00", "type": "user", "content": "..."}
{"timestamp": "2026-04-07T10:01:00", "type": "assistant", "content": "..."}
```

Nachher (Markdown + YAML)

```
---
session_id: 2026-04-07-session-001
timestamp_start: 2026-04-07T10:00:00
timestamp_end: 2026-04-07T10:30:00
agent: CodeAnalyzer
tags: [refactoring, rust]
---

# Session: Code-Analyse

## User (10:00)
Bitte analysiere die Datei atomic.rs

## Assistant (10:01)
Die Datei enthaelt folgende Funktionen...
```

6.3 Auto-Commit

Nach jeder Session automatisch:

```
git add .
git commit -m "Session 2026-04-07: [Agent] [Zusammenfassung]"
git push
```

7. Phase 3: Obsidian RAG

3 OFFEN

7.1 Ziel

Semantic Search ueber den gesamten Obsidian Vault mittels Vektor-Embeddings.

7.2 Technologie-Stack

Tabelle 6 RAG-Komponenten

Komponente	Technologie	Zweck
Embeddings	BGE-M3	Multilinguale Embeddings, 8192 Tokens
Vector DB	LanceDB	Lokale Vektor-Datenbank
Chunking	Markdown-Aware	Intelligentes Splitting von Notizen
Python-Layer	sentence-transformers	Embedding-Generierung

7.3 Workflows

1. **Indexierung:** Alle Markdown-Dateien einlesen, chunken, embedden, in LanceDB speichern
2. **Query:** Nutzerfrage embedden, aehnliche Chunks suchen, Kontext an LLM senden
3. **Update:** Bei neuen/veraenderten Dateien Delta-Indexierung

8. Phase 4: Code-Analyse

4 OFFEN

8.1 Ziel

Multi-Agent-System fuer Code-Analyse mit Tree-sitter Parsing.

8.2 Agent-System

Tabelle 7 Spezialisierte Agents

Agent	Zustaendigkeit	Input	Output
CodeAnalyzer	Code-Struktur, Komplexitaet	Source Code	AST, Metriken
DocumentationAgent	Doku-Qualitaet, Luecken	Code + Docs	Doku-Report
SecurityAgent	Sicherheitsprobleme	Source Code	Security-Report
RefactorAgent	Refactoring-Vorschlaege	Code + Metriken	Refactor-Plan

8.3 Tree-sitter Integration

```
// Parser-Initialisierung
let parser = Parser::new();
parser.set_language(tree_sitter_rust::LANGUAGE.into())?;

// AST generieren
let tree = parser.parse(code, None)?;
let root = tree.root_node();

// Queries fuer Patterns
let query = Query::new(
    tree_sitter_rust::LANGUAGE.into(),
    "(function_item) @func"
)?;
```

9. Phase 5: Sequential Workflow

5 OFFEN

9.1 Ziel

Orchestrierung der Agents in sequentieller Reihenfolge (NICHT parallel - Hardware-Limit).

9.2 Workflow-Beispiel

```
User Request: "Analysiere diesen Code"
    |
    v
Orchestrator empfaengt Request
    |
    v
Phase 1: CodeAnalyzer
    - Parsed Code mit Tree-sitter
    - Erstellt AST + Metriken
    |
    v
Phase 2: SecurityAgent
    - Bekommt AST + Code
    - Prueft auf Security-Issues
    |
    v
Phase 3: RefactorAgent
    - Bekommt alle vorherigen Ergebnisse
    - Erstellt Refactoring-Vorschlaege
    |
    v
Orchestrator aggregiert Ergebnisse
    |
    v
Output an User
```

9.3 Kontext-Weitergabe

Jeder Agent bekommt:

- Original-Request

- Alle Ergebnisse vorheriger Agents
- RAG-Kontext aus Obsidian Vault

10. Phase 6: Integration & Polishing

6 OFFEN

10.1 Ziel

Fertige CLI und Vorbereitung fuer Web-Service.

10.2 CLI-Features

```
openclaw session start                    # Neue Session starten
openclaw session list                     # Alle Sessions anzeigen
openclaw session convert <file>           # JSONL zu Markdown
openclaw vault index                      # RAG-Index aktualisieren
openclaw vault search "<query>"           # Semantic Search
openclaw analyze <path>                   # Code-Analyse starten
openclaw agent list                       # Verfuegbare Agents
openclaw config set <key> <value>         # Konfiguration
```

10.3 Abstraction Layer

Interface-Pattern fuer austauschbare Implementierungen:

```
trait FileSystem {
    fn write(&self, path: &Path, content: &str) -> Result<()>;
    fn read(&self, path: &Path) -> Result;
}

struct LocalFileSystem;
struct RemoteFileSystem;  // Fuer spaeteren Web-Service
```

11. Zusammenfassung & Naechste Schritte

11.1 Was wurde erreicht

Tabelle 8 Projekt-Status

Bereich	Status	Details
Phase 0: Stabilisierung	100%	Atomic Writes, 23 Dateien repariert
Phase 1: Modularisierung	0%	Crate-Struktur planen
Phase 2: State Management	0%	Markdown + YAML Konvertierung
Phase 3: Obsidian RAG	0%	LanceDB + BGE-M3 Setup
Phase 4: Code-Analyse	0%	Tree-sitter + Agents
Phase 5: Workflow	0%	Orchestrator implementieren
Phase 6: Integration	0%	CLI + Web-Service

11.2 Naechste Schritte

Sofort (Phase 1 beginnen)

1. Neue Crates erstellen: `openclaw-agents`, `openclaw-rag`, `openclaw-parser`
2. Workspace-Struktur erweitern
3. Bestehende Code-Teile in Crates verschieben

Kurzfristig (Phase 2-3)

1. Session-Konvertierung JSONL zu Markdown implementieren
2. YAML-Frontmatter Generator
3. LanceDB Setup fuer lokale Vektor-Datenbank
4. BGE-M3 Embeddings Integration

11.3 Wichtige Dateien & Pfade

Tabelle 9 Wichtige Pfade

Beschreibung	Pfad
Repository	https://github.com/andre-bubel/openclaw
Lokaler Workspace	C:\Users\andre\.openclaw\workspace
Rust Binary	target\release\openclaw-core.exe
PowerShell Module	workspace\scripts\Safe-FileWrite.psm1
Backups (pro Ordner)	.backups*.backup.*.ps1
Geloeschte Dateien	.deleted*

11.4 Kontakt & Dokumentation

Dieser Plan wurde am 7. April 2026 erstellt. Die Master-Dokumentation befindet sich im Repository unter `MASTER_SUMMARY_OPENCLAW_RENOVATION.md`.

> **Hinweis fuer zukuenftige Sessions**
>
> Bei neuen LLM-Sessions diesen Plan als Kontext verwenden. Aktueller Stand: Phase 0 abgeschlossen, Phase 1 als naechstes.